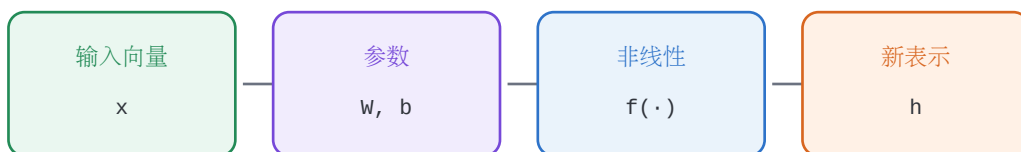


第4讲：神经网络到底是什么？

参数、层、激活函数与“学习”的最小直觉

从一条公式 $h = f(Wx+b)$ ，建立理解大型语言模型的神经网络底座



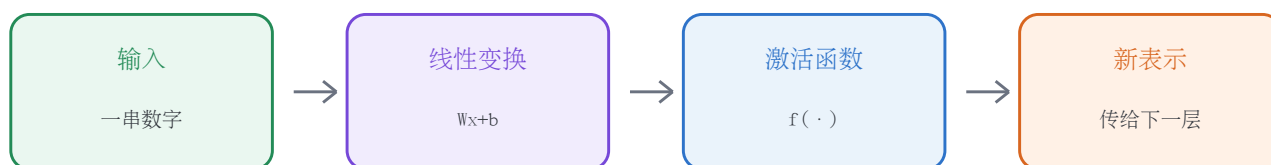
适合对象：零基础。不要求先学微积分，也不要求会 PyTorch。

本讲重点：理解“前向计算”发生了什么；梯度与反向传播留到下一讲系统展开。

本讲学习目标

核心目标 学完以后，你应该能用自己的话解释：神经元在算什么；权重、偏置、参数、层、激活函数分别是什么；为什么很多层要配合非线性；“几千亿参数”究竟指什么；训练与推理有何区别；Transformer 为什么仍然属于神经网络。

知识地图



层次	本讲要求
直觉层	把神经网络理解成“很多可调旋钮控制的数值变换系统”，而不是神秘的大脑复制品。
公式层	看懂 $h=f(Wx+b)$ ，知道 W 和 b 是参数， x 是输入， h 是输出/新表示。
结构层	理解神经元、层、隐藏层、深度、宽度与多层组合。
能力层	理解非线性激活为什么让网络能表示复杂函数。
LLM 层	知道 Embedding、Attention、FFN 等都在更大的神经网络计算图中工作。

1

先把“神经网络”去神秘化

本质：给数字做一系列可学习的变换。

很多人第一次听到“人工神经网络”，会自然联想到人脑神经元。这个历史类比有帮助，但如果把它当成真实机制反而容易误解。现代神经网络首先是一个[数学与计算系统](#)：输入一组数字，通过一组参数控制的运算，得到另一组数字。

对 LLM 来说，上一讲得到的 Token Embedding 正好就是这些“输入数字”。因此链路继续变成：



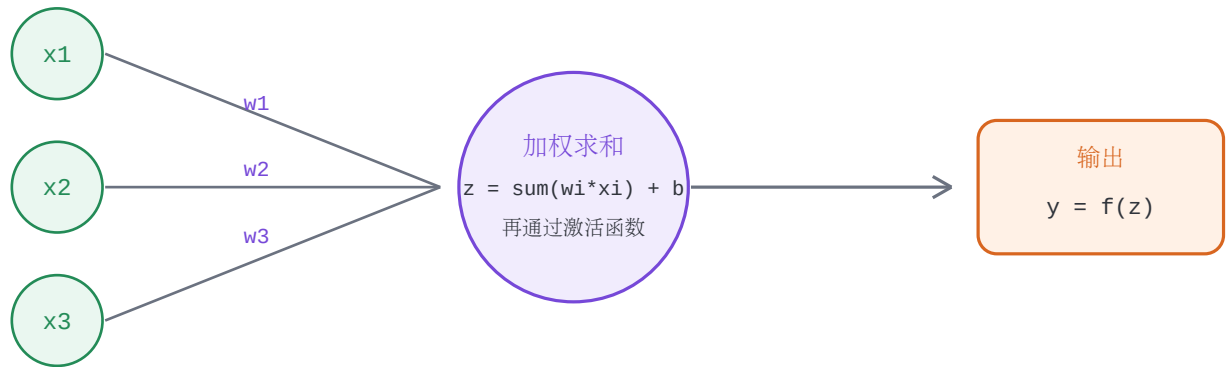
最小定义 神经网络可以先理解成：由许多带可学习参数的函数层层组合起来形成的函数。模型训练的核心，就是找到一组更合适的参数。

因此，大模型里所谓“模型学到了什么”，从可操作层面看，就是训练以后模型内部的大量参数取到了某些数值，使它在大量输入上产生更有用的输出。

2

一个神经元到底算什么？

输入 $\times$ 权重 $\rightarrow$ 相加 $+$ 偏置 $\rightarrow$ 激活函数。



一个“神经元”并不神秘：输入数字 $\times$ 权重 $\rightarrow$ 相加 $+$ 偏置 $\rightarrow$ 激活函数 $\rightarrow$ 输出数字。

假设有三个输入： x_1 、 x_2 、 x_3 。每个输入都有一个权重 w_1 、 w_2 、 w_3 。神经元先计算：

$$z = w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + b$$

然后再通过一个激活函数 f ：

$$y = f(z)$$

这里每个符号承担不同角色：

符号	直觉	是否通常需要学习
x	输入数据 / 上一层传来的表示	不是模型参数
w	权重；决定某个输入对结果影响有多大	是
b	偏置；让变换可以整体平移	是
f	激活函数；引入非线性	函数形式通常是设计选择
y	当前神经元/层的输出	由计算产生

通俗比喻 把 w 看成调音台上每一路声音的音量旋钮， b 像整体基准偏移， f 像一个非线性效果器。训练就是自动调这些旋钮。

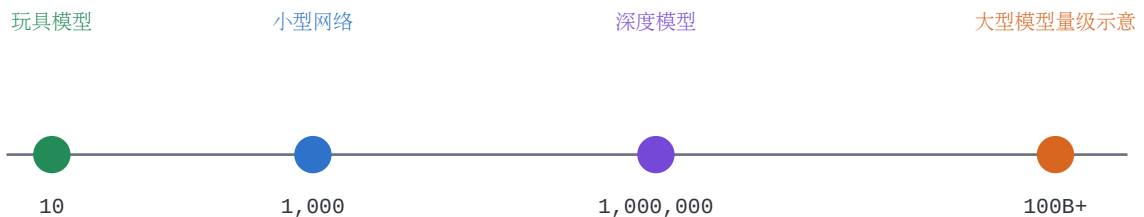
3

权重、偏置、参数：三者到底什么关系？

“参数”是总称，权重和偏置是最典型的参数。

你以后会频繁看到“7B、70B、数千亿参数”。这里的 parameter（参数）不是“设置菜单里的 temperature=0.7”那种参数，而是模型训练过程中被学习出来并保存在模型文件里的数值。

“参数量”是模型中可学习数字的总数，不等于知识条数、神经元数或训练数据条数。



横轴仅为概念示意，不按比例；不同模型架构的参数规模不同。

概念	例子	训练时会不会改变
模型参数 Parameter	权重矩阵 W、偏置 b、Embedding Matrix 等	会，通过优化算法更新
超参数 Hyperparameter	学习率、batch size、层数、hidden size 等	通常由研究者/训练配置决定，不靠普通反向传播直接学习
推理配置	temperature、top-p、max tokens 等	不属于模型已训练权重

关键误区 “模型有 100B 参数”不等于“模型存了 100B 条知识”，也不等于“有 100B 个神经元”。参数是一大批浮点数/量化数值，它们共同决定函数行为。

4

一层神经网络为什么常写成矩阵？

真实模型不会一个神经元一个神经元手算，而是把很多运算并行成矩阵乘法。

如果一层里有很多输出单元，与其写几百条 $w_1 \times x_1 + w_2 \times x_2$ ，不如把所有权重排成一个矩阵 W 。于是整层可以写成：



$$h = f(Wx + b)$$

如果输入 x 有 4,096 个数字，输出也有 4,096 个数字，那么 W 可以是 $4,096 \times 4,096$ 的矩阵。仅这一个矩阵就有：

$$4096 \times 4096 = 16,777,216$$

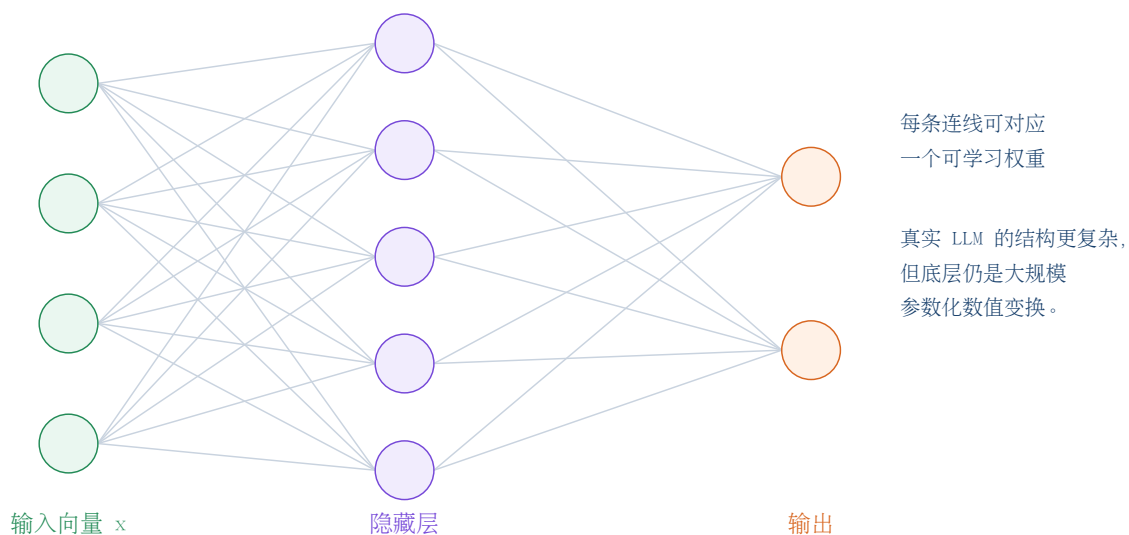
约 1,677 万个权重。现代 GPU / TPU 最擅长的事情之一，就是并行执行这种大规模矩阵运算。

为什么 AI 需要 GPU LLM 的核心计算里有大量矩阵乘法。GPU 拥有大量并行计算单元和高带宽显存，特别适合一次处理海量乘加运算；所以“大模型=很多矩阵运算”是理解 AI 硬件生态的一条主线。

5

什么叫“层”？什么叫“深度”？

一层把一种表示变成另一种表示；多层就是函数的连续复合。



上图是最经典的教学画法。真实 Transformer 不会只是一排圆点，但“上一层表示 $\rightarrow$ 参数化变换 $\rightarrow$ 下一层表示”的思想完全一致。

术语	通俗解释
Input / 输入	进入某个模块的数值向量。
Hidden Layer / 隐藏层	位于输入与最终输出之间的中间计算层；“隐藏”不是神秘，只是用户不直接看到。
Output / 输出	当前网络或模块最后给出的数值。
Depth / 深度	通常指顺序堆叠了多少计算层/块；不同论文统计口径可能不同。
Width / 宽度	一层内部表示有多宽，例如 hidden size。
Hidden State	某一层、某一位置正在携带的内部表示。

论文阅读提醒 “32 layers” 在 LLM 论文里常常指 32 个 Transformer blocks，而不是把每一个矩阵乘法都单独算作一层。遇到 layer 数量要先看作者的架构定义。

6

为什么不能只做线性变换？

如果没有非线性，堆很多层在表达能力上会严重退化。

只堆线性层

$$x \rightarrow W_1x \rightarrow W_2(W_1x)$$

$$= (W_2 W_1) x$$

多层仍可合并成一个线性变换

加入非线性激活

$$x \rightarrow f(W_1x+b_1) \rightarrow f(W_2h+b_2)$$

组合后可表达更复杂的函数

激活函数的核心价值：打破“无论叠多少层都只是线性”的限制。

假设没有激活函数：

$$\begin{aligned} h_1 &= W_1 x \\ h_2 &= W_2 h_1 \\ &= W_2 W_1 x \\ &= W_{\text{total}} x \end{aligned}$$

无论你堆多少层线性变换，最终仍然可以合并成一个更大的线性变换。这样网络很难表达现实世界复杂的弯曲、分段和条件关系。

核心原因

激活函数让“层与层的组合”不再能简单合并成一个矩阵，从而显著增加模型可表示的函数复杂度。

这也是“深度学习”里的“深”真正有意义的前提之一：不是单纯层数多，而是多层非线性变换能逐步构造复杂表示。

7

常见激活函数：ReLU、GELU、SiLU

不需要背公式，但要知道它们在网络里扮演“非线性门”的角色。

函数	简单直觉	常见情况
ReLU	负数变 0，正数保留： $\max(0, x)$	经典深度学习中非常常见
GELU	比 ReLU 更平滑，不是硬切断	Transformer/LLM 中长期常见
SiLU / Swish	x 乘一个平滑门控值	现代 LLM/视觉模型中也很常见
Sigmoid	把值压到 $0 \sim 1$	门控、概率式输出等场景；深层主干中不一定首选

以 ReLU 为例：

$x = -2 \rightarrow \text{ReLU}(x) = 0$
 $x = 0 \rightarrow \text{ReLU}(x) = 0$
 $x = 3 \rightarrow \text{ReLU}(x) = 3$

不要死记 你现在不需要记每个激活函数的精确曲线。先掌握一条：激活函数引入非线性，使多层网络能表达更复杂关系。后面读具体架构时再关注 GELU/SiLU 的细节。

8

一次“前向计算”到底发生了什么？

模型拿到输入以后，从前到后算一遍，这叫 forward pass。

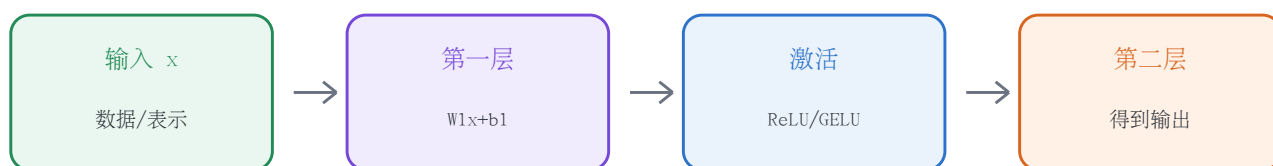
假设一个极小网络只有两层：

$x = [1.0, 2.0]$

$h1 = \text{ReLU}(W1 @ x + b1)$

$y = W2 @ h1 + b2$

程序做的事情就是：读取参数 $\rightarrow$ 做乘法与加法 $\rightarrow$ 经过激活 $\rightarrow$ 把结果继续送进下一层。这个过程叫 Forward Pass（前向传播/前向计算）。



在 LLM 推理时，这种前向计算会发生在很多 Transformer block、很多 token 位置上。最终模型把最后的 hidden state 转换成整个词表上的 logits / probabilities，用于预测下一个 token。

重要连接 你前面学的 Embedding 本身就是神经网络参数的一部分；本讲的矩阵 W 、偏置 b 只是另外一些参数。LLM 不是“Embedding + 一个神秘黑箱”，而是大量参数化模块串起来。

9

训练与推理：不要混成一件事

训练负责改参数；推理主要用已学参数进行前向计算。

训练 Training

输入样本

→ 前向计算得到预测

→ 计算 Loss

→ 反向传播得到梯度

→ 更新参数 W , b , Embedding...

推理 Inference

参数通常固定

输入新数据

→ 前向计算

→ 得到输出 / 下一个 token 概率

不会每次回答都重写全部权重

学习发生在参数更新；使用模型回答问题通常属于推理。

很多初学者会问：“我和 ChatGPT 聊了十分钟，它是不是就把我的内容训练进模型参数了？”从机制上必须先区分：**普通一次推理**与**模型训练/微调**不是同一种操作。

训练需要一个目标、Loss、梯度和参数更新过程；普通推理通常只是读取当前参数做前向计算。产品层面可能另有日志、记忆、后续训练流程，但那是系统机制，不能和“当前这次前向推理自动修改全部权重”混为一谈。

下一讲预告 Loss 到底是什么？梯度怎么告诉参数“该往哪个方向改”？Backpropagation 为什么能把错误一路传回前面？这些会在第5讲系统解释。

10

“学习”到底是什么意思？先建立最小直觉

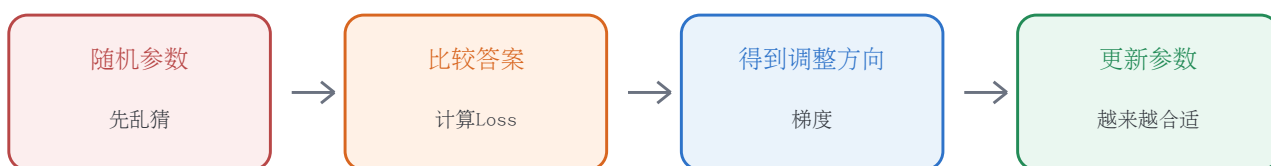
模型不是被逐条写规则，而是通过数据把参数调到更合适的位置。

假设我们想让一个极小模型学会：输入越大，输出也越大。最开始随机参数可能给出很差的预测：

```
■■ x=2
■■■■=4
■■■■ w=0.3, b=0.1
■■ y = 0.3*2 + 0.1 = 0.7
```

预测 0.7 与正确答案 4 相差很大。训练算法会根据误差，判断 w 、 b

应该如何微调。经过很多样本、很多轮更新，参数可能逐渐靠近能得到正确规律的一组值。



学习的可操作定义

在神经网络中，“学习”通常可以落实为：根据训练目标，让可学习参数不断更新，使 Loss 下降并提高对目标任务的表现。

需要警惕：Loss 下降不自动意味着“真正理解”“事实可靠”“道德正确”。它只说明模型更符合训练目标与数据中的某些统计规律。

11

一个参数到底“存”了什么？

通常不能把单个权重解释成一条完整知识。

初学者常会想：“模型知道巴黎是法国首都，那一定有某个参数专门存着这句话。”真实情况通常不是这样。现代深度网络的表示往往是**分布式 (distributed)** 的：一个行为/概念可能依赖许多参数共同作用，一个参数也可能参与许多不同模式。

可以类比一张非常复杂的调音台：某首歌的最终声音不是由一个旋钮决定，而是许多旋钮共同作用；同一个旋钮也影响很多声音成分。

错误直觉	更准确的理解
“1个参数 = 1条知识”	知识与能力通常分散在大量参数、激活和层间交互中。
“参数越多 = 一定越聪明”	参数规模只是容量条件之一，还受数据、训练方法、架构、推理机制等影响。
“模型文件就是文本数据库”	模型文件主要保存数值权重，不是把训练网页原样按目录存储。
“知道参数值就能直接读懂模型思想”	数十亿高维参数的含义通常无法逐个用自然语言解释。

科研连接 这也是 mechanistic interpretability（机制可解释性）为什么困难：研究者想从分布式表示和复杂电路中找出“哪些组件共同实现了某种行为”。

12

神经网络和 Transformer 是什么关系？

Transformer 不是神经网络的替代品，而是一种神经网络架构。

从层级关系看，可以这样理解：



Transformer block 里面的 Attention、Feed-Forward Network (FFN/MLP)、LayerNorm、Residual Connection 等模块共同工作。其中 FFN 本身就是典型的“线性层 $\rightarrow$ 激活函数 $\rightarrow$ 线性层”结构；Attention 也大量依赖可学习矩阵与矩阵乘法。

Token IDs

- > Embedding (■■■)
- > Transformer Block $\times$ N
 - > Attention: $W_q, W_k, W_v, W_o \dots$
 - > FFN/MLP: $W_1, W_2, \dots + \text{activation}$
- > Output projection
- > next-token logits

因此 LLM 所谓几十亿、几百亿参数，主要来自 Embedding、Attention 投影矩阵、FFN/MLP 矩阵、输出层等大量可学习数值的总和。

13

为什么 LLM 里的 FFN/MLP 很重要？

Attention 负责“看哪里”，前馈网络负责对每个位置的表示做强力变换。

你还没正式学 Attention，但可以先知道 Transformer block 里通常有两类非常核心的计算：

模块	可以先怎么理解	本讲关联
Self-Attention	让一个 token 根据其他 token 的信息更新自己	大量矩阵 $W_q/W_k/W_v/W_o$
FFN / MLP	对每个 token 的 hidden state 做非线性变换	最直接对应 $h=f(Wx+b)$ 的多层版本
Residual / Norm	稳定信息流与训练	后续专门讲

很多 LLM 中，FFN 中间维度会比 hidden size 更宽，因此其权重矩阵会贡献非常可观的参数量与计算量。这说明你今天学的“普通神经网络层”并没有过时，它就藏在 Transformer 核心结构里。

学习路线提醒 理解 Attention 之前先理解矩阵层、参数、激活和前向计算，会让后面的 Q/K/V 少很多神秘感。Attention 的本质也不是“模型突然有眼睛”，仍然是结构化的数值计算。

技术史：神经网络为什么经历多次起伏？

从感知机到深度学习，再到 Transformer。

时间	节点	为什么重要
1943	McCulloch & Pitts 提出早期人工神经元模型	把神经元抽象成可计算逻辑单元的早期尝试。
1957-1958	Frank Rosenblatt 的 Perceptron（感知机）	展示简单可学习分类器，引发早期热潮。
1969	感知机局限受到系统讨论	单层线性模型能力有限，领域进入低潮之一。
1980s	反向传播在多层网络训练中重新受到重视	让多层网络的参数更新变得实际可用。
2012	AlexNet 在 ImageNet 上取得显著成果	GPU + 大数据 + 深度网络推动深度学习爆发。
2017	Transformer 提出	以 Attention 为核心的架构成为后来 LLM 的关键基础。

历史给出的启示 神经网络能力不是只靠某一个公式突然出现，而是算法、数据、算力、工程和规模共同推进的结果。现代 LLM 仍然建立在这些基础数值运算之上。

15

常见误区

这些误解会直接妨碍后面理解 Transformer。

误区1：人工神经网络就是在精确模拟人脑。

现代神经网络受神经科学启发，但主要是工程化数学模型，不能简单等同于生物大脑。

误区2：一个参数就是一条知识。知识和功能通常分布在大量参数及其交互中。

误区3：网络层数越多一定越强。

深度需要合适的架构、优化、数据与训练稳定性；乱堆层可能更难训练。

误区4：只要有很多线性层就能自动变复杂。没有非线性时，多层线性变换可合并成一个线性变换。

误区5：推理时模型会像训练一样自动改写全部参数。

普通推理主要固定参数做前向计算；训练需要专门的 Loss、反向传播和优化步骤。

误区6：参数多就必然知识更多、推理更强。

参数量只是影响能力的一个因素，数据、训练、架构、后训练与推理策略同样关键。

16

动手实验：手工跑一个两层小网络

不安装 NumPy/PyTorch，直接看清每一步前向计算。

材料包里的 `04_tiny_neural_network_demo.py` 只使用 Python 标准库。它完成三件事：

1) 定义一个小型线性层；2) 实现 ReLU；3) 让一个二维输入通过两个线性层，打印每一步中间结果。

```
x = [1.0, 2.0]

# Layer 1
z1 = W1 @ x + b1
h1 = relu(z1)

# Layer 2
y = W2 @ h1 + b2
```

建议你运行后做两个改动：

- ① 把 `W1` 中某个权重从 0.5 改成 5.0，观察最终输出怎样变化。
- ② 把 ReLU 临时去掉，再比较网络输出。

实验目的 不是让你“学会写神经网络框架”，而是亲眼确认：所谓神经网络前向传播，底层就是数字、乘法、加法、激活函数和层层传递。

17

本讲术语表

以后看到这些词，要知道它属于“结构、参数、计算还是训练”。

术语	中文直觉	你应该会解释什么
Neural Network	神经网络	许多参数化函数组成的计算系统。
Weight (W)	权重	决定输入如何影响输出的可学习数值。
Bias (b)	偏置	在线性变换后增加的可学习偏移。
Parameter	参数	训练会更新并保存在模型中的可学习数值。
Hyperparameter	超参数	由训练设计/配置决定的设置，如学习率、层数。
Layer	层	把一组表示变成另一组表示的计算模块。
Hidden State	隐藏状态	网络内部某层产生的中间表示。
Activation Function	激活函数	引入非线性，如 ReLU/GELU/SiLU。
Forward Pass	前向计算	从输入一路计算到输出。
Training	训练	根据 Loss/梯度更新参数。
Inference	推理	使用训练好的参数对新输入做前向计算。
MLP / FFN	多层感知机/前馈网络	Transformer 中的重要非线性子网络。

18

练习题：检查是否真正形成了“参数化函数”直觉

答案在独立练习册中。

1. 为什么说 Token ID 本身不能直接作为语义，而 Embedding 可以成为神经网络输入？结合前一讲回答。
2. 在 $y=f(wx+b)$ 中， x 、 w 、 b 、 f 、 y 分别是什么角色？哪些通常是可学习参数？
3. 为什么“100B 参数”不等于“100B 条知识”？
4. 如果一个线性层输入维度是 100，输出维度是 50，仅权重矩阵 W 有多少参数？如果再加 bias 呢？
5. 为什么不加激活函数时，连续两个线性层可以合并？这会带来什么限制？
6. Forward Pass 与 Training 的区别是什么？
7. 为什么普通一次聊天推理不能简单说成“模型刚刚又训练了一次”？
8. Transformer 为什么仍然属于神经网络？至少列出两个内部的可学习矩阵来源。
9. “参数量越大模型一定越强”哪里不严谨？
10. 用自己的话解释：神经网络的“学习”在可操作层面到底发生了什么。

过关标准 如果你能不看材料解释清楚：输入向量 $\rightarrow Wx+b \rightarrow$ 激活 $\rightarrow$ 多层组合 $\rightarrow$ 输出，并说清训练负责“改参数”、推理负责“用参数”，这一讲就已经达标。

19

一页总结：把整讲压成八句话

这八句话是进入 Loss / Backpropagation 的桥。

1. 神经网络首先是数学计算系统，不是人脑的精确复制。
2. 一个最小神经元可以写成 $y=f(\sum w_ix_i+b)$ 。
3. 权重和偏置是典型可学习参数；参数量就是这类可学习数字的总数。
4. 一整层通常写成向量/矩阵形式 $h=f(Wx+b)$ ，以便并行计算。
5. 多层网络把表示逐层变换；隐藏状态就是中间层正在携带的表示。
6. 激活函数引入非线性，否则很多线性层最终仍等价于一个线性变换。
7. 训练通过 Loss、梯度、优化器更新参数；推理通常固定参数做前向计算。
8. Transformer 仍是神经网络：Embedding、Attention、FFN 等都由大量参数化矩阵运算组成。

与下一讲的连接

现在还剩一个最大的黑箱：模型怎么知道哪个参数应该调大、哪个应该调小？

下一讲进入第5讲《模型是怎么“学习”的？——Loss、梯度、反向传播与梯度下降》。

